AI Content Studio

Clone & Fresh-Instance Guide — stand up a new copy with a new user and separate brands, carrying **zero** prior data.

Generated from `docs/clone-guide.html` · re-render with `docs/render-pdf.sh`.

1 · What this platform is made of

The whole studio is four layers. Cloning cleanly means knowing which layer to copy, which to drop, and which to regenerate.

Layer	What lives there	On a new box
Git repo `aicontentstudio/`	All code: `compose.yml`, `plugins/studio`, `dashboard/`, `renderer/`, `scripts/`, the four sub-app stacks, the seeded occasions calendar.	**CLONE — this *is* the platform.**
SQLite store `studio-data/`	`studio.db` (jobs, drafts, brands, learnings, campaigns, media, settings) + `knowledge/` & `bm-config/` (Basic-Memory index) + logs.	**DON'T COPY — `db.init_db()` rebuilds the schema on first run; occasions re-seed; brands start empty by design.**
Docker volumes	Each sub-app's own DB: `postiz_*` (connected socials, schedule, analytics), `chatwoot_*` (inbox, contacts), `typebot_*` (flows), `mautic_*` (contacts, campaigns), `basic-memory-config`.	**DON'T COPY — fresh stacks = empty apps; the new user re-onboards their own accounts.**
Env files `.env` ×2	Root `.env` + `~/.hermes/.env`. All secrets **and** the operator's identity (bot tokens, admin login, Telegram allow-list, OIDC keys).	**TEMPLATE + REGENERATE — copy the shape, replace identity/secrets (§5).**

> **The mental model**
>
> Carry the **code**. Drop the **data** (SQLite + volumes). Regenerate the **identity** (.env). That is the entire trick — everything below is detail.

2 · Prerequisites on the new box

Requirement	Notes
Docker + Docker Compose v2	`docker compose` (not the old `docker-compose`).
The `claude` CLI, logged in	The Studio "brain" (research, writing, image prompts, the vision fit-check) runs through the **host's `claude` subscription** via a bridge — **not** an API key. Install it and run an interactive login once so the session is valid. See §7 (this is the easiest thing to forget).
git	To clone the repo.
~4 vCPU / 16 GB RAM / 80 GB disk	The 1-vCPU/8 GB original was too small (video render pegged the box). 4 vCPU / 16 GB is comfortable. Video/worker still run one job at a time by design (§8).
A Linux user (e.g. `hermes`)	Match `HERMES_UID` / `HERMES_GID` in `.env` to this user so bind-mounted files are owned correctly. Add the user to the `docker` group.

3 · Step-by-step clone

1. **Clone the repo** to the new box (push this repo to a git remote first if it isn't on one):

   ```
   git clone <your-remote> aicontentstudio
   cd aicontentstudio
   ```

2. **Create the two `.env` files** from the templates and regenerate identity/secrets — see the §5 checklist:

   ```
   # root env (studio + sub-apps)
   cp .env.example .env            # if no example exists, copy your old .env and blank the §5 rows
   # Hermes's own env (Telegram allow-list lives here, NOT in root .env)
   nano ~/.hermes/.env             # set TELEGRAM_ALLOWED_USERS to the NEW operator
   ```

3. **Do NOT copy** `studio-data/` or any `*_*` Docker volume from the old box. Starting without them is what guarantees a clean slate.
4. **Bring up the studio**, then each sub-app stack:

   ```
   docker compose up -d                      # hermes, dashboard, renderer, basic-memory
   (cd postiz   && docker compose up -d)
   (cd chatwoot && docker compose up -d)
   (cd typebot  && docker compose up -d)
   # Mautic is OPTIONAL and currently DISABLED — skip it. The content engine has zero
   # dependency on it (plugins/ = 0 refs); the dashboard Funnels area just no-ops. Re-enable
   # later with: (cd mautic && docker compose up -d) and un-hide the Funnels nav item.
   ```

   The first worker tick runs `db.init_db()` → fresh `studio.db` with the full schema and a re-seeded occasions calendar.

5. **Install the cron block** (§6) with `crontab -e`. This drives the worker/scout/heartbeat and starts the two host bots on boot. **Adjust the paths** if the repo or home dir differ.

6. **Set the VM to survive reboots** (if it's a VM): on the hypervisor, `qm set <vmid> --onboot 1` and install `qemu-guest-agent` in the guest, so a host power event auto-restarts everything.
7. **Log in & configure**: open the dashboard (`http://<box-ip>:4008`), log in with the new `DASH_USER`, then under **Settings** re-enter the ~17 preferences (studio name, region, ZAR rate, model seams, etc.) and add your brands fresh.
8. **Re-onboard the sub-apps**: connect the new social accounts in Postiz, create the Chatwoot inbox, build Typebot flows. Copy the resulting IDs/tokens back into `.env` (the §5 "re-created on onboarding" rows) and `docker compose up -d` to pick them up.
9. **Smoke-test** with §9 before handing it over.

4 · Service map (what comes up)

Stack	Services	Reach
Studio (`compose.yml`)	hermes (host-net), dashboard, renderer, basic-memory	Dashboard `:4008` · MCP `127.0.0.1:8808`
Postiz (`postiz/`)	postiz, postgres, redis, temporal (+ es, pg, ui), admin-tools	Publishing + scheduling
Chatwoot (`chatwoot/`)	rails, sidekiq, postgres, redis	§3d engagement inbox
Typebot (`typebot/`)	builder, viewer, db, redis, mailpit, proxy	Funnel flows (/flows)
Mautic (`mautic/`) optional / off	mautic, mautic-cron, db	Nurture / broadcast — heavy; currently disabled, Funnels nav hidden. Skip unless needed.
Host processes (cron `@reboot`): `claude-bridge.py` on `127.0.0.1:4014` · `nancy_bot.py` (Telegram).		

5 · The `.env` regeneration checklist

When the new instance is for a **different operator**, do not reuse the identity fields. Three buckets:

REGENERATE — new identity & secrets, per instance

Key	How to get a fresh value
`TELEGRAM_BOT_TOKEN` , `NANCY_BOT_TOKEN`	New bots via @BotFather.
`TELEGRAM_ALLOWED_USERS` (in ~/.hermes/.env), `TELEGRAM_CHAT_ID`	The new operator's Telegram user/chat id.
`DASH_USER` (+ password)	New admin login (password is set/changed in Settings; stored hashed in the DB).
`OPERATOR_EMAIL`	New operator's email.
`SESSION_SECRET` , `CLAUDE_BRIDGE_TOKEN` , `POSTIZ_JWT_SECRET` , `CHATWOOT_WEBHOOK_TOKEN` , `FUNNEL_CAPTURE_TOKEN`	Fresh random strings: `openssl rand -hex 32` .
`STUDIO_OIDC_CLIENT_ID` , `STUDIO_OIDC_CLIENT_SECRET` , `STUDIO_OIDC_ISSUER` , `STUDIO_OIDC_REDIRECT_URIS`	New issuer = the new box URL; new client id/secret; redirect URIs point at the new host.
`STUDIO_COCKPIT_URL` , `HEALTHCHECK_URL`	New box's LAN/host URL; a new healthcheck ping URL.

RE-CREATED ON ONBOARDING — fill after the new user sets up each sub-app

Key	Source
`POSTIZ_API_KEY`, `POSTIZ_API_URL`, `POSTIZ_USER_ID`	From the new Postiz account.
`CHATWOOT_API_TOKEN`, `CHATWOOT_ACCOUNT_ID`	From the new Chatwoot account.
`MAUTIC_API_USER`, `MAUTIC_API_PASSWORD`, `MAUTIC_API_URL`	From the new Mautic install.

REUSE OR SWAP — shared AI / infra keys (not identity)

Key	Note
`FAL_KEY`, `FAL_VIDEO_MODEL`	Image/video generation (fal). Reuse or give the new user their own.
`OPENROUTER_API_KEY`, `TAVILY_API_KEY`	Social-pulse reasoning & worker web search.
`ELEVENLABS_API_KEY`, `ELEVENLABS_VOICE_ID`, `ELEVENLABS_VOICE_ID_RU`	Video voiceover.
`STUDIO_TEXT_MODEL`, `STUDIO_TEXT_PROVIDER`	Studio text-gen model seam.
`ZAR_PER_USD`, `CHATWOOT_DEFAULT_BRAND`	Locale / default routing.
`HERMES_UID`, `HERMES_GID`, `HERMES_IMAGE_TAG`	Match the new box's user; pin the image version.

6 · Cron block (copy-paste, then fix paths)

The scheduled jobs drive the pipeline; the two `@reboot` lines start the host bots. The token values are read out of `.env` at boot.

```
# --- scheduled work ---
*/5  * * * * /home/hermes/aicontentstudio/scripts/heartbeat.sh  >>
/home/hermes/aicontentstudio/studio-data/heartbeat.log 2>&1
*/2  * * * * /home/hermes/aicontentstudio/scripts/run-worker.sh >>
/home/hermes/aicontentstudio/studio-data/worker.log 2>&1
*/15 * * * * /home/hermes/aicontentstudio/scripts/run-scout.sh  >>
/home/hermes/aicontentstudio/studio-data/scout.log 2>&1
30   * * * * /home/hermes/aicontentstudio/scripts/disk-watch.sh >>
/home/hermes/aicontentstudio/studio-data/disk-watch.log 2>&1

# --- host bots on boot ---
# NOTE: CLAUDE_BIN is REQUIRED. cron's PATH is minimal, so the bridge cannot find
# the `claude` binary without it -> the whole Claude brain silently falls back.
@reboot CLAUDE_BRIDGE_TOKEN=$(grep '^CLAUDE_BRIDGE_TOKEN=' /home/hermes/aicontentstudio/.env |
cut -d= -f2-) CLAUDE_BIN=/home/hermes/.local/bin/claude /usr/bin/python3
/home/hermes/aicontentstudio/scripts/claude-bridge.py >> /home/hermes/aicontentstudio/studio-
data/claude-bridge.log 2>&1
@reboot NANCY_BOT_TOKEN=$(grep '^NANCY_BOT_TOKEN=' /home/hermes/aicontentstudio/.env | cut -d= -
f2-) TELEGRAM_ALLOWED_USERS=$(grep '^TELEGRAM_ALLOWED_USERS=' /home/hermes/.hermes/.env | cut -
d= -f2-) STUDIO_REPO=/home/hermes/aicontentstudio CLAUDE_BIN=/home/hermes/.local/bin/claude
/usr/bin/python3 /home/hermes/aicontentstudio/scripts/nancy_bot.py >>
/home/hermes/aicontentstudio/studio-data/nancy.log 2>&1
```

Find the binary path on the new box with `which claude` and substitute it for `/home/hermes/.local/bin/claude`.

7 · The Claude brain (don't skip this)

Research, copywriting, image prompts, and the "does the rendered image actually match the brief" **vision fit-check** all go through `scripts/claude-bridge.py` on `127.0.0.1:4014`, which shells out to the host's `claude` CLI under a **subscription login** — there is no Anthropic API key in `.env`.

On a new box you must: (1) install the `claude` CLI, (2) run it once interactively to log in, (3) ensure the cron line passes `CLAUDE_BIN` (§6). Verify with the end-to-end ping in §9 — if it returns an `Errno 2 'claude'` error, the binary isn't on the bridge's PATH; if it returns an auth error, the subscription needs re-login. Either way the studio keeps running with a graceful non-Claude fallback, but quality drops, so confirm it before handoff.

8 · How the worker runs (so testers aren't surprised)

Cron fires `run-worker.sh` every 2 minutes → `worker.py --once`. It takes a single lockfile and processes **exactly one job per tick**; concurrent ticks log `"another run is in progress, skipping"` — that is **normal**, not an error. One-at-a-time is a deliberate race-safety choice (a long run must not outlive the stale-lock window), not just an old single-CPU limit. The queue drains one job per free tick; a thorough

research+draft+media run can take several minutes. Raising true parallelism is a real change (per-slot locks) best done *after* the test round.

9 · Pre-handoff smoke test

Run these on the new box; all should be green before you give it to a tester.

```
# 1. all studio containers up
docker compose ps

# 2. each sub-app stack up
for d in postiz chatwoot typebot mautic; do (cd $d && docker compose ps); done

# 3. host bots running (exactly one python each, no duplicates)
pgrep -af '[p]ython.*nancy_bot.py'
pgrep -af '[p]ython.*claude-bridge.py'

# 4. DB integrity + queue health (no stuck 'processing' jobs)
docker compose exec -T hermes python3 -c "import
sqlite3;c=sqlite3.connect('/opt/studio/studio.db');\
print('integrity:',c.execute('PRAGMA integrity_check').fetchone()[0]);\
print('stuck:',c.execute(\"SELECT COUNT(*) FROM jobs WHERE
queued_action='processing'\").fetchone()[0])"

# 5. dashboard responds
curl -s -o /dev/null -w 'dashboard %{http_code}\n' http://127.0.0.1:9119/

# 6. Claude bridge is alive AND reaches Claude (THE key check)
curl -s http://127.0.0.1:4014/health           # -> {"ok": true}
TOK=$(grep '^CLAUDE_BRIDGE_TOKEN=' .env | cut -d= -f2-)
curl -s -X POST http://127.0.0.1:4014/ask -H 'Content-Type: application/json' \
  -H "X-Bridge-Token: $TOK" \
  -d '{"prompt":"Reply with exactly the three words: bridge check ok","timeout":90}'
# expect: {"text": "bridge check ok"}

# 7. disk headroom
df -h /
```

10 · Optional: factory-reset a clone in place

> **Destructive — wipes all studio data and sub-app databases.**
>
> Only run on a clone you intend to blank, never on a live instance. Back up first.

```
# stop everything
docker compose down
for d in postiz chatwoot typebot mautic; do (cd $d && docker compose down -v); done  # -v wipes
sub-app volumes

# drop the studio data (schema rebuilds on next start)
rm -f  studio-data/studio.db studio-data/studio.db-* studio-data/*.log
rm -rf studio-data/knowledge studio-data/bm-config

# blank the identity rows in .env (§5 REGENERATE), then bring it back up
docker compose up -d
```

For a brand-new operator on a brand-new box, prefer the §3 fresh-clone path over reset-in-place — it leaves nothing behind to clean up.
